

# Docker 实操题对照临摹版

针对考卷第 45、46 题，提供标准答案 + 常见错误对照 + 逐行解析

## 第 45 题：Dockerfile 标准答案

### ✅ 标准写法

```
# 1. 基础镜像（必须大写，冒号后指定版本标签）
FROM python:3.11-alpine

# 2. 设置工作目录（后续 COPY/RUN 都基于此路径）
WORKDIR /app

# 3. 先复制依赖文件，利用缓存层优化
# 只要 requirements.txt 不变，这一层就不会重新构建
COPY requirements.txt .

# 4. 安装依赖（--no-cache-dir 避免缓存，减小镜像层）
RUN pip install --no-cache-dir -r requirements.txt

# 5. 复制应用代码（.dockerignore 要排除不需要的文件）
COPY . .

# 6. 暴露端口（文档性质，实际映射用 -p 参数）
EXPOSE 5000

# 7. 创建非 root 用户（安全最佳实践）
RUN adduser -D appuser

# 8. 切换到非 root 用户运行
USER appuser

# 9. 容器启动命令（JSON 数组格式，可被 docker run 覆盖）
CMD ["python", "app.py"]
```

### ❌ 你的错误对照

| 你的写法                            | 问题                  | 正确写法                                                                          |
|---------------------------------|---------------------|-------------------------------------------------------------------------------|
| from python:3.11-alpine         | 指令必须大写              | FROM python:3.11-alpine                                                       |
| workdir /app                    | 指令必须大写              | WORKDIR /app                                                                  |
| copy ./requirements.txt<br>（缺了） | 缺目标路径，指令小写<br>没安装依赖 | COPY requirements.txt .<br>RUN pip install --no-cache-dir -r requirements.txt |
| （缺了）<br>grouproot:no            | 没复制应用代码<br>不存在这个指令  | COPY . .<br>RUN adduser -D appuser + USER appuser                             |
| cmd["python", "app.py "]        | 指令小写，格式错误，多余空格      | CMD ["python", "app.py"]                                                      |

## 📌 关键知识点

### 为什么先 COPY requirements.txt 再 COPY . . ?

Docker 构建是分层缓存机制:

- 如果 requirements.txt 没变 → 直接用缓存层, 不重新 pip install
- 如果代码变了 (.py 文件) → 只重新 COPY . . 这一层, 依赖层不动
- 反过来如果先 COPY . . , 任何代码改动都会触发重新 pip install

### 为什么用非 root 用户?

- 安全: 容器被攻破后, 攻击者只有 appuser 权限, 不是宿主机 root
- 合规: 很多公司安全基线要求容器不能以 root 运行
- Alpine 创建用户: adduser -D appuser (-D 表示不设置密码)

### CMD vs ENTRYPOINT 怎么选?

```
# CMD 可被覆盖 (适合应用启动命令)
CMD ["python", "app.py"]
# docker run myimage python manage.py migrate ← 可以覆盖

# ENTRYPOINT 固定入口 (适合工具类镜像)
ENTRYPOINT ["python"]
CMD ["app.py"]
# docker run myimage manage.py migrate ← python 固定, 后面参数替换 CMD
```

## 第 46 题: docker-compose.yml 标准答案

### ✅ 标准写法

```
# 版本声明 (新版 Docker 可省略, 但建议写上)
version: '3.8'

# 服务定义 (核心块, 所有容器配置在这里)
services:
  # 服务名: web
  web:
    # 使用的镜像
    image: nginx:alpine
    # 端口映射: 宿主机 80 → 容器 80
    ports:
      - "80:80"
    # 卷挂载: 宿主机路径 → 容器路径
    volumes:
      - ./html:/usr/share/nginx/html
    # 依赖: 等 db 启动后再启动 web
    depends_on:
      - db
    # 加入自定义网络
    networks:
      - app-net
```

```
# 服务名: db
db:
  image: mysql:8.0
  # 环境变量
  environment:
    MYSQL_ROOT_PASSWORD: "123456"
  # named volume 挂载
  volumes:
    - db-data:/var/lib/mysql
  networks:
    - app-net

# 卷声明 (named volume 必须在这里定义)
volumes:
  db-data:
    # driver: local # 默认就是 local, 可省略

# 网络声明 (自定义网络必须在这里定义)
networks:
  app-net:
    driver: bridge # 默认就是 bridge, 可省略
```

✖ 你的错误对照

| 你的写法                  | 问题                         | 正确写法                                       |
|-----------------------|----------------------------|--------------------------------------------|
| name web              | 不是关键字, 应该是 services: 下的服务名 | services: web:                             |
| from nginx:alpine     | 关键字错误                      | image: nginx:alpine                        |
| prot: 8080:80         | 拼写错误, 端口错误                 | ports: - "80:80"                           |
| mount: ./html:...     | 关键字错误                      | volumes: - ./html:/usr/share/nginx/html    |
| server:               | 关键字错误                      | services:                                  |
| name:db               | 缩进错误, 应该是 services 的子项     | db: (缩进和 web 同级)                           |
| images:mysql:8.0      | 关键字错误 (多了 s)               | image: mysql:8.0                           |
| env:MYSQL_ROOT...     | 关键字错误                      | environment: MYSQL_ROOT_PASSWORD: "123456" |
| volume: db-data:...   | 关键字错误, 位置错误                | volumes: - db-data:/var/lib/mysql          |
| yilai:db              | 拼音, 关键字错误                  | depends_on: - db                           |
| network: name:app-net | 结构错误                       | networks: app-net: driver: bridge          |
| (缺了)                  | 没声明 volumes 和 networks 块   | 文件末尾必须有 volumes: 和 networks:               |

📌 关键知识点

yaml 格式铁律:

```
# 1. 缩进必须是 2 个空格（不能用 Tab）
services:
  web:          # ← 2 空格缩进
    image: nginx # ← 4 空格缩进
    ports:      # ← 4 空格缩进
      - "80:80" # ← 6 空格缩进（- 算一个缩进层级）

# 2. 冒号后面必须有空格
key: value      # ✔️
key:value       # ❌

# 3. 字符串建议加引号（特别是带特殊字符的）
ports:
  - "80:80"     # ✔️ 带冒号，必须引号
environment:
  PASS: "123456" # ✔️ 纯数字字符串，建议引号
```

### depends\_on 的真相:

```
# depends_on 只是控制启动顺序，不保证服务就绪
# db 容器启动了，但 MySQL 可能还在初始化，web 直接连会报错

# 生产级做法：web 服务加健康检查或重试机制
services:
  web:
    depends_on:
      db:
        condition: service_healthy # 等 db 健康后才启动
  db:
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 5s
      timeout: 3s
      retries: 5
```

### 端口映射 vs EXPOSE:

```
# docker-compose.yml 里的 ports 是实际映射
ports:
  - "80:80"      # 宿主机 80 → 容器 80

# Dockerfile 里的 EXPOSE 只是文档说明，不实际映射
EXPOSE 5000      # 告诉别人这个容器会监听 5000，但外部访问不到
```

### named volume vs bind mount 在 compose 中的区别:

```
volumes:
  # bind mount: 宿主机具体路径
  - ./html:/usr/share/nginx/html

  # named volume: Docker 管理，可跨容器共享，可备份
```

```
- db-data:/var/lib/mysql

# named volume 必须在文件底部声明
volumes:
  db-data:      # 空对象表示使用默认配置
```

## 临摹练习步骤

### 步骤 1：抄写标准答案

把上面的标准 Dockerfile 和 compose 文件手打一遍，不要复制粘贴。

### 步骤 2：验证语法

```
# 验证 Dockerfile (在 Dockerfile 所在目录)
docker build -t test:v1 .

# 验证 docker-compose.yml
docker compose config
# 如果输出正常，说明 yaml 语法正确
# 如果有报错，根据提示修正缩进或关键字
```

### 步骤 3：运行测试

```
# 测试 Dockerfile
docker run -d -p 5000:5000 --name test-app test:v1
docker logs test-app

# 测试 docker-compose
docker compose up -d
docker compose ps
docker compose logs -f
curl http://localhost      # 测试 nginx 是否通
```

### 步骤 4：故意制造错误，看报错

```
# 试试把 image 写成 from，看 compose 报什么错
# 试试缩进用 Tab，看报什么错
# 试试 ports 写成 prot，看报什么错
```

## 面试话术模板

### 被问到 Dockerfile 编写要点时：

“我通常遵循这几个原则：第一，选择合适的基础镜像，比如 Alpine 减小体积；第二，合理利用分层缓存，把不常变动的指令放前面，比如先 COPY requirements.txt 再安装依赖；第三，安全方面使用非 root 用户运行；第四，多阶段构建分离编译环境和运行环境，进一步减小镜像。”

**被问到 docker-compose 时：**

“我用 compose 编排多容器应用，主要配置 services 定义各个容器，volumes 做数据持久化，networks 实现容器间通信。比如一个 Web + DB 的场景，web 服务映射端口、挂载静态文件，db 服务用 named volume 持久化数据，两个服务通过自定义网络互通，web 通过 depends\_on 确保等 db 启动后再启动。”

打印出来，对照标准答案，手敲 3 遍，直到能独立写出无错版本。